학습데이터처리

데이터 살펴보기

© KOPO-Dept.
AI Engineering Limited

데이터 살펴보기

- 데이터 프레임의 구조
  - UCI 자동차 연비(auto mpg) Dataset
    - 연비, 실린더 수, 배기량, 출력, 차중, 가속능력 등에 관한 데이터 398 개로 구성(Row: 398, Column 9)

No.	속성(attributes)		데이터 상세(범위)
1	mpg	연비	연속 값
2	cylinders	실린더 수	이산 값(예시: 3, 4, 6, 8)
3	displacement	배기량	연속 값
4	horsepower	출력	연속 값
5	weight	차중	연속 값
6	acceleration	가속능력	연속 값
7	model_year	출시년도	이산 값(예: 70, 71, 80, 81)
8	origin	제조국	이산 값(예: 1(USA), 2(EU), 3(JPN))
9	name	모델명	문자열

[표 3-1] UCI 데이터셋 – "auto mpg" 상세 항목

데이터 살펴보기

- 데이터 프레임의 구조
  - UCI 자동차 연비(auto mpg) Dataset 내용 미리 보기
    - **head()**: 정수 n을 입력하면 처음 n개의 행을 보여줌 (Default n: 5)
    - **tail()**: 정수 n을 입력하면 마지막 n개의 행을 보여줌 (Default n: 5)

```
• 앞부분 미리보기: DataFrame 객체.head(n)
• 뒷부분 미리보기: DataFrame 객체.tail(n)
```

```
2  18.0          8           ...                 1          plymouth satellite
3  16.0          8           ...                 1               amc rebel sst
4  17.0          8           ...                 1                 ford torino

[5 rows x 9 columns]


      mpg   cylinders      ...         origin                name
393  27.0           4      ...              1  ford mustang gl
394  44.0           4      ...              2         vw pickup
395  32.0           4      ...              1       dodge rampage
396  28.0           4      ...              1          ford ranger
397  31.0           4      ...              1          chevy s-10

[5 rows x 9 columns]
```

데이터 살펴보기

- 데이터 프레임의 구조
  - 데이터 요약 정보 확인하기
    - 데이터 프레임의 크기(행, 열): **shape**
      - 데이터프레임 클래스의 shape 속성은 행과 열의 개수를 튜플 형태로 반환

```
데이터프레임의 크기 확인: DataFrame 객체.shape
```

    - 데이터 프레임의 기본 정보: **info()**
      - 데이터 프레임에 대한 기본 정보 출력: 클래스 유형, 행 인덱스의 구성, 열 이름의 종류와 개수, 메모리 할당량

```
데이터프레임의 기본 정보 출력: DataFrame 객체.info()
```

```
cylinders       398 non-null int64
displacement    398 non-null float64
horsepower      398 non-null object
weight          398 non-null float64
acceleration    398 non-null float64
model year      398 non-null int64
origin          398 non-null int64
name            398 non-null object
dtypes: float64(4), int64(3), object(2)
memory usage: 24.9+ KB
None
```


© KOPO-Dept.
AI Engineering Limited

데이터 살펴보기

- 데이터 프레임의 구조
  - 데이터 요약 정보 확인하기
    - 데이터 프레임의 기술(Description) 통계 정보 요약: **describe()**
      - **산술 데이터를 갖는 열에 대한 주요 기술 통계 정보(평균, 표준편차, 최소/최대/중간 값 등)를 요약해서 출력**
      - 산술 데이터가 아닌 열에 대한 정보도 보고 싶을 때, include='all'

데이터프레임의 기술 통계 정보 요약: `DataFrame 객체.describe()`

	mpg	cylinders	...	model year	origin
count	398.000000	398.000000	...	398.000000	398.000000
mean	23.514573	5.454774	...	76.010050	1.572864
std	7.815984	1.701004	...	3.697627	0.802055
min	9.000000	3.000000	...	70.000000	1.000000
25%	17.500000	4.000000	...	73.000000	1.000000
50%	23.000000	4.000000	...	76.000000	1.000000
75%	29.000000	8.000000	...	79.000000	2.000000
max	46.600000	8.000000	...	82.000000	3.000000

[8 rows x 7 columns]

	acceleration	model year	origin	name
count	398.000000	398.000000	398.000000	398
unique	NaN	NaN	NaN	305
top	NaN	NaN	NaN	ford pinto
freq	NaN	NaN	NaN	6
mean	15.568090	76.010050	1.572864	NaN
std	2.757689	3.697627	0.802055	NaN

데이터 살펴보기

- 데이터 프레임의 구조
  - 데이터 개수 확인하기
    - 각 열의 데이터 개수 확인: **count()**
      - 데이터프레임의 각 열이 가지고 있는 데이터 개수를 시리즈 객체로 반환
      - 유효한 값의 개수만을 계산

데이터프레임의 기술 통계 정보 요약: `DataFrame 객체.` **count()**

```
mpg             398
cylinders       398
displacement    398
horsepower      398
weight          398
acceleration    398
model year      398
origin          398
name            398
dtype: int64


<class 'pandas.core.series.Series'>
```

데이터 살펴보기

- 데이터 프레임의 구조
  - 데이터 개수 확인하기
    - 각 열의 고유값 개수 확인: **value_count()**
      - 시리즈 객체의 고유값(unique value) 개수를 반환 (각 열의 고유값의 종류와 개수 확인 가능)
      - 고유값이 행 인덱스가 되고, 고유값의 개수가 데이터 값이 되는 시리즈 객체가 만들어 짐.
      - dropna옵션: True (NaN을 카운팅 x)

```
열 데이터의 고유값 개수: DataFrame 객체["열 이름"].value_counts()
```

df['origin'].value_counts() 결과
- 'origin': 제조국가 (USA:1 ➔ 249개)

```
1    249
3     79
2     70
Name: origin, dtype: int64


<class 'pandas.core.series.Series'>
```

데이터 살펴보기

- 통계함수 적용
  - 평균값: **mean()**
    - 산술 데이터를 갖는 모든 열의 평균값을 각각 계산하여 시리즈 객체로 반환
      - 데이터프레임의 특정 열의 평균값을 계산 할 수도 있음

```
• 모든 열의 평균값: DataFrame 객체.mean()
• 특정 열의 평균값: DataFrame 객체["열 이름"].mean()
```

```
mpg              23.514573
cylinders         5.454774
displacement    193.425879
weight         2970.424623
acceleration     15.568090
model year       76.010050
origin            1.572864
dtype: float64
```

데이터 살펴보기

- 통계함수 적용
  - 중간값: **median()**
    - 산술 데이터를 갖는 모든 열의 중간값을 계산하여 시리즈로 반환
      - 데이터프레임의 특정 열의 평균값을 계산 할 수도 있음

```
• 모든 열의 중간값: DataFrame 객체.median()
• 특정 열의 중간값: DataFrame 객체["열 이름"].median()
```

```
mpg              23.0
cylinders         4.0
displacement    148.5
weight         2803.5
acceleration     15.5
model year       76.0
origin            1.0
dtype: float64


23.0
```

데이터 살펴보기

- 통계함수 적용
  - 최대/최소값: **max/min()**
    - 데이터 프레임의 각 열이 갖는 데이터 값 중에서 최대/최소 값을 계산하여 시리즈로 반환
    - 데이터프레임의 특정 열의 최대/최소값을 계산 할 수도 있음
      - 문자열 데이터를 가진 열(name, horsepower)는 **문자열을 ASCII로 변환하여 크고 작음을 비교**

- 모든 열의 최대값: `DataFrame 객체.max()`
- 특정 열의 최대값: `DataFrame 객체["열 이름"].max()`

- 모든 열의 최소값: `DataFrame 객체.min()`
- 특정 열의 최소값: `DataFrame 객체["열 이름"].min()`

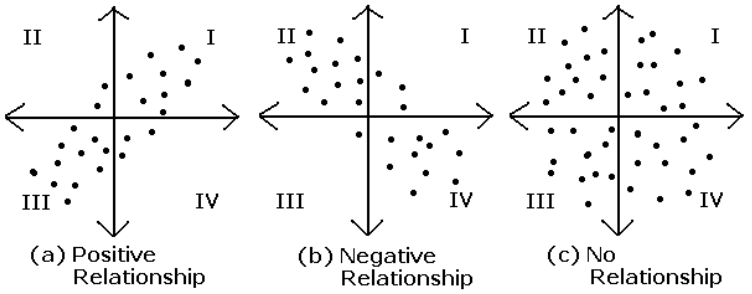
```
mpg                      46.6
cylinders                   8
displacement              455
horsepower                  ?
weight                   5140
acceleration             24.8
model year                 82
origin                      3
name           vw rabbit custom
dtype: object


46.6
```

데이터 살펴보기

- Appendix
  - 편차, 분산, 표준편차
    - 표준편차: 평균에 대한 오차
      - **평균 값이 실제 데이터 값으로부터 얼마나 오차(오류)가 있는지를 알기 위해** 사용
    - 편차: 원래 값에서 평균을 뺀 값(편차는 +도 될 수 있고, -도 될 수 있음)
      - 편차들의 합을 통해서 오차를 구할 수 있음 (양수, 음수가 나오기 때문에 제곱 사용 필요)
    - 분산: 편차의 제곱의 평균 (양수화)
    - 분산 vs. 표준편차

  - 공분산, 상관계수
    - 두 변수 간의 관계 표현 (평균, 분산은 하나의 변수에 대한 분포를 표현하는 좋은 방법)
    - 공분산 $Cov(X, Y) = E((X - \mu)(Y - \nu))$
      - Cov(X,Y) > 0 : X가 증/감 ➔ Y도 증/감
      - Cov(X,Y) < 0: X가 증/감 ➔ Y는 감/증
      - Cov(X,Y) == 0: 두 변수간에는 **선형 관계**가 없음



데이터 살펴보기

- Appendix
  - 공분산, 상관계수
    - 공분산의 경우, 두 변수의 단위의 크기에 영향을 받음
      - 100점 만점인 두 과목의 공분산 (상관성이 없더라도 큰 값이 나올 수 있음(vs. 10점))
      - 10점 만점인 두 과목의 공분산(상관성이 크더라도 작은 값이 나옴 (vs. 100점))
    - 이를 보완하기 위해 상관계수
      - 변수의 절대적 크기에 영향을 받지 않도록 분산의 크기만큼 나누었다고 생각하면 됨.

$$\rho = \frac{Cov(X,\ Y)}{\sqrt{Var(X)\ \ Var(Y)}}, \quad -1 \leq \rho \leq 1$$

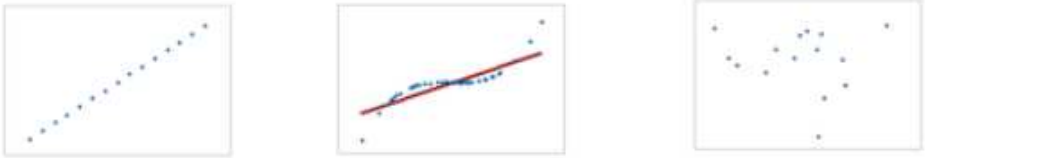
© KOPO-Dept.
AI Engineering Limited

데이터 살펴보기

- Appendix
  - 상관계수
    - Pearson 상관계수: 두 변수 간에 선형성이 얼마나 강한지를 측정하기 위해 사용
      - 선형성이 강하다는 것은 변수들의 관계가 직선에 의해 잘 모델링된다는 의미
    - Spearman 상관계수: 단조성을 평가하기 위해 사용
      - 단조성이 좋다(1에 가까울수록)는 것은 한 변수의 값의 크기가 커지면(또는 작아지면) 다른 변수의 크기도 커짐(작아짐)

$$PLCC=\frac{\sum_{i=1}^{M}(X_i-\overline{X})(Y_i-\overline{Y})}{\sqrt{\sum_{i=1}^{M}(X_i-\overline{X})^2(Y_i-\overline{Y})^2}}$$

$$SROCC=1-\frac{6}{M(M^2-1)}\sum_{i=1}^{M}d_i^2$$



Pearson = +1, Spearman = +1 | Pearson = +0.851, Spearman = +1 | Pearson = −0.093, Spearman = −0.093

그림출처: 미니탭

데이터 살펴보기

- 통계함수 적용
  - 표준편차: **std()**
    - 산술 데이터를 갖는 열의 표준편차를 계산하여 시리즈로 반환
    - 데이터프레임의 특정 열을 선택하여 계산 할 수도 있음

- 모든 열의 표준편차: `DataFrame 객체.std()`
- 특정 열의 표준편차: `DataFrame 객체["열 이름"].std()`

```
mpg              7.815984
cylinders        1.701004
displacement   104.269838
weight         846.841774
acceleration     2.757689
model year       3.697627
origin           0.802055
dtype: float64


7.815984312565782
```

데이터 살펴보기

- 통계함수 적용
  - 상관계수: **corr()**
    - 산술 데이터를 갖는 모든 열에 대하여 2개씩 서로 짝을 짓고, 각각의 경우에 대하여 상관계수를 계산
      - 상관계수로는 Pearson, kendall, Spearman (Default: Pearson)
      - 방법을 지정 하고 싶을 경우: df.corr(method="spearman")으로 지정 가능

```
• 모든 열의 상관계수: DataFrame 객체.corr()
• 특정 열의 상관계수: DataFrame 객체[열 이름의 리스트].corr()
```

	mpg	cylinders	...	model year	origin
mpg	1.000000	-0.775396	...	0.579267	0.563450
cylinders	-0.775396	1.000000	...	-0.348746	-0.562543
displacement	-0.804203	0.950721	...	-0.370164	-0.609409
weight	-0.831741	0.896017	...	-0.306564	-0.581024
acceleration	0.420289	-0.505419	...	0.288137	0.205873
model year	0.579267	-0.348746	...	1.000000	0.180662
origin	0.563450	-0.562543	...	0.180662	1.000000

[7 rows x 7 columns]

Correlation를 계산하는 방법
- 한 컬럼과 다른 모든 컬럼들의 Correlation (corrwith)

- 두 컬럼 이상 간의 Correlation (corr)

```
df.corrwith(df['avg'], method='spearman') # 'avg' 컬럼과 다른 모든 컬럼간의 Spearman Rank Correlation
df['avg'].corr(df[5], method='kendall') # 'avg' 컬럼과 5 컬럼 간의 Kentall Tau Correlation
df.corr(method='pearson') # 모든 컬럼 간의 Pearson Correlation
```

데이터 살펴보기

- 실습

```python
import pandas as pd

# read_csv() 함수로 df 생성
df = pd.read_csv('./auto-mpg.csv', header=None)

# 열 이름을 지정
df.columns = ['mpg','cylinders','displacement','horsepower','weight',
              'acceleration','model year','origin','name']

# 데이터프레임 df의 내용을 일부 확인
print(df.head())      # 처음 5개의 행
print('\n')
print(df.tail())      # 마지막 5개의 행

# 모든 열을 출력
pd.set_option('display.max_columns', None)
# 모든 행을 출력
# pd.set_option('display.max_rows', None)
# # 행 출력의 개수를 조절
# pd.options.display.max_rows = 60

# df의 모양과 크기 확인: (행의 개수, 열의 개수)를 튜플로 반환
print(df.shape)
```

```
    mpg  cylinders  displacement  ... model year  origin                       name
0  18.0          8         307.0  ...         70       1  chevrolet chevelle malibu
1  15.0          8         350.0  ...         70       1          buick skylark 320
2  18.0          8         318.0  ...         70       1         plymouth satellite
3  16.0          8         304.0  ...         70       1              amc rebel sst
4  17.0          8         302.0  ...         70       1                ford torino

[5 rows x 9 columns]
```

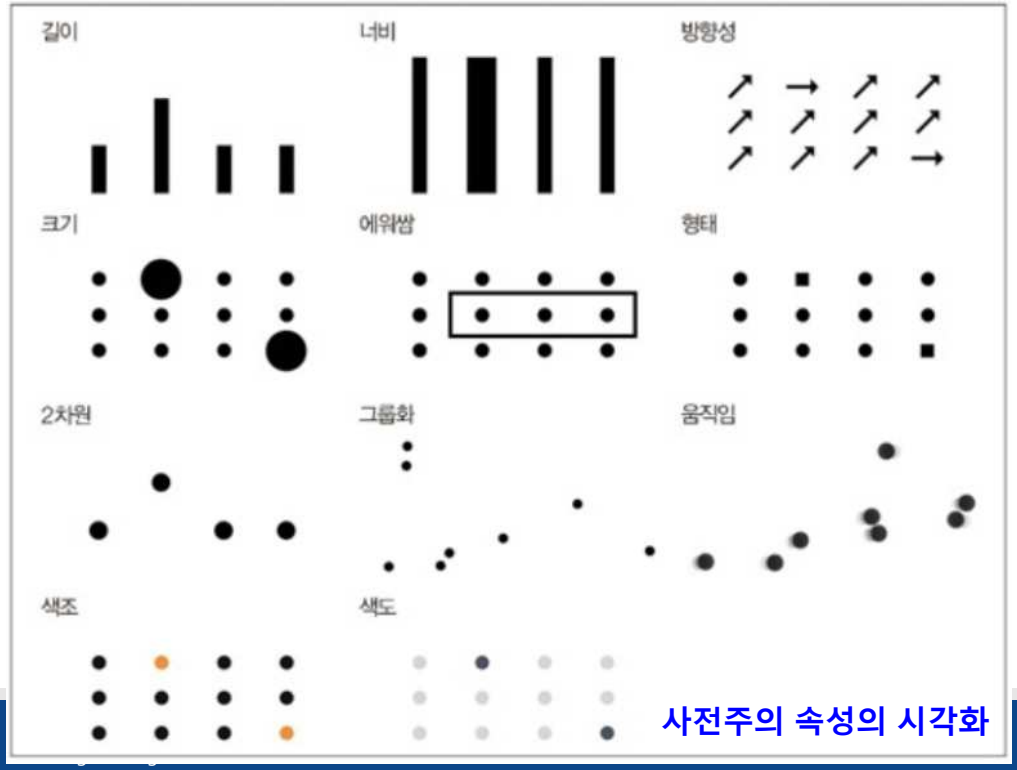
학습데이터처리

데이터 살펴보기-Graph

© KOPO-Dept.
AI Engineering Limited

데이터 살펴보기

- 데이터 **시각화의 필요성**
  - 사전주의 속성
    - **보고 있는 것을 이해하기 위해 [생각하거나 의식적으로 노력할 필요없이 시각적 이미지의 특징 및 패턴을 알아볼 수 있는 능력]을 의미**
  - 데이터 시각화는 **메시지 전달을 위하여 사전주의 속성을 사용**



사전주의 속성의 시각화

데이터 살펴보기

- 데이터 **시각화의 필요성**
  - 이 표에서 가장 큰 수는?
  - 얼마나 많은 수의 매장이 한 달 판매 목표치인 450대를 넘어섰는가?

표1. 영국의 상반기 자전거 판매량

자전거매장	1월	2월	3월	4월	5월	6월
런던	358	118	769	636	196	293
요크	557	404	533	115	989	309
리즈	564	540	112	633	606	961
멘체스터	525	278	196	596	119	693
버밍업	509	378	325	699	592	927

인지부하(Cognitive load): 정보를 보고 해석하는데 사용하는 시간 및 노력

© KOPO-Dept.
AI Engineering Limited

데이터 살펴보기

- 데이터 **시각화의 필요성**
  - 이 표에서 가장 큰 수는? 989

표1. 영국의 상반기 자전거 판매량

자전거매장	1월	2월	3월	4월	5월	6월
런던	358	118	769	636	196	293
요크	557	404	533	115	**989**	309
리즈	564	540	112	633	606	961
멘체스터	525	278	196	596	119	693
버밍업	509	378	325	699	592	927

색의 변화를 이용하여 가장 높은 자전거 판매량 강조

© KOPO-Dept.
AI Engineering Limited

데이터 살펴보기

- 데이터 **시각화의 필요성**
  - 얼마나 많은 수의 매장이 한 달 판매 목표치인 450대를 넘어섰는가? 17

표1. 영국의 상반기 자전거 판매량

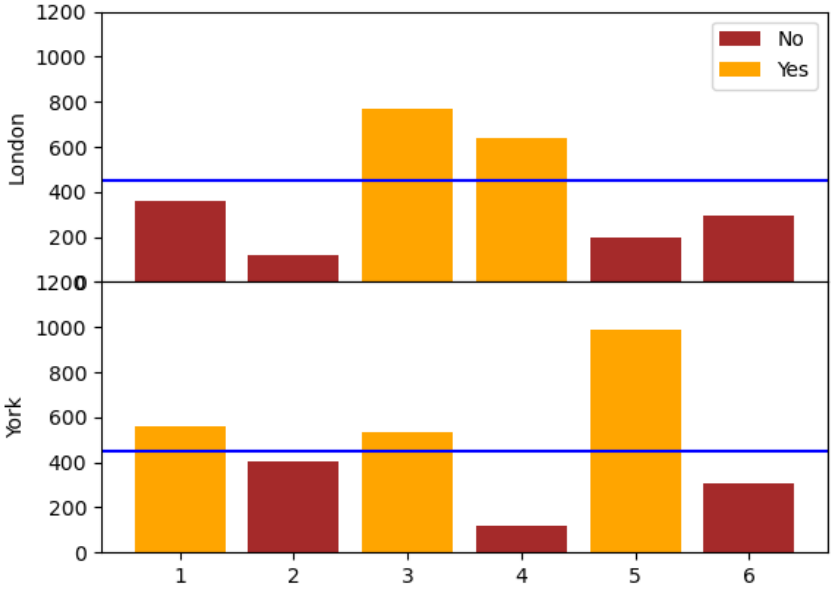
자전거매장	1월	2월	3월	4월	5월	6월
런던	358	118	**769**	**636**	196	293
요크	**557**	404	**533**	115	**989**	309
리즈	**564**	**540**	112	**633**	**606**	**961**
멘체스터	**525**	278	196	**596**	119	**693**
버밍업	**509**	378	325	**699**	**592**	**927**

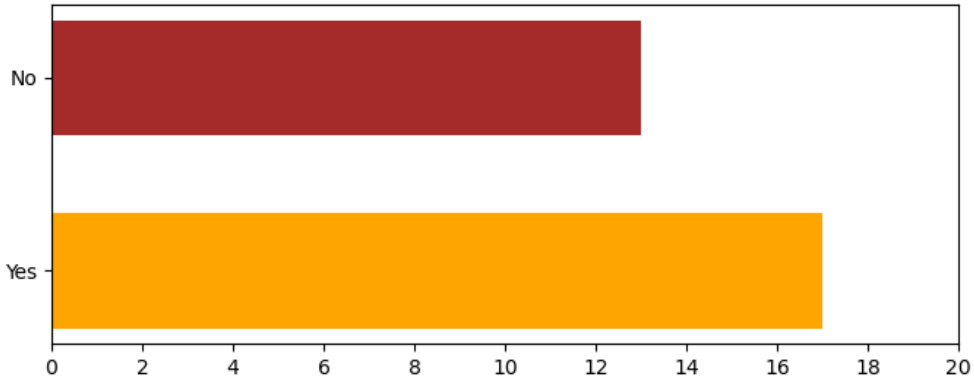
색의 변화를 이용한 판매 목표치보다 큰 값 색으로 강조

© KOPO-Dept.
AI Engineering Limited

데이터 살펴보기

- 데이터 **시각화의 필요성**
  - 얼마나 많은 수의 매장이 한 달 판매 목표치인 450대를 넘어섰는가? 17
  - 그래프로 표현





© KOPO-Dept.
AI Engineering Limited

데이터 살펴보기

- 판다스 내장 그래프 도구 활용
  - 그래프를 이용한 시각화 방법은 데이터의 분포와 패턴을 파악하는데 크게 도움이 됨
    - 시각화를 위한 별도 라이브러리 존재(matplotlib, seaborn 등), 간단하게 분석하는 용도로 내장 그래프 사용
  - 시리즈 또는 데이터프레임 객체에 **plot() 메소드**를 적용하고 **kind 옵션**으로 그래프의 종류를 선택

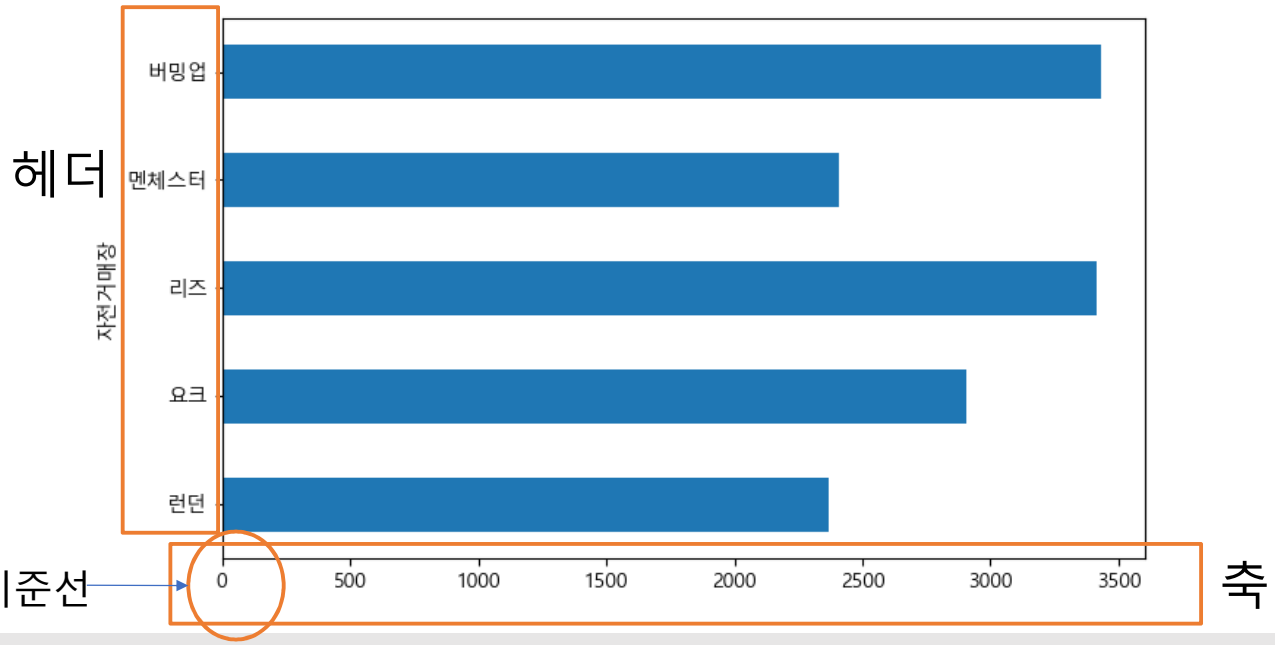
kind 옵션	설명	kind 옵션	설명
'line'	선 그래프	'kde'	커널 밀도 그래프
'bar'	수직 막대 그래프	'area'	면적 그래프
'barh'	수평 막대 그래프	'pie'	파이 그래프
'his'	히스토그램	'scatter'	산점도 그래프
'box'	박스 플롯	'hexbin'	고밀도 산점도 그래프

```
선 그래프: DataFrame 객체.plot()
```

Plot() 메소드 적용시, 옵션을 추가하지 않으면, Default는 선그래프(line)

데이터 살펴보기

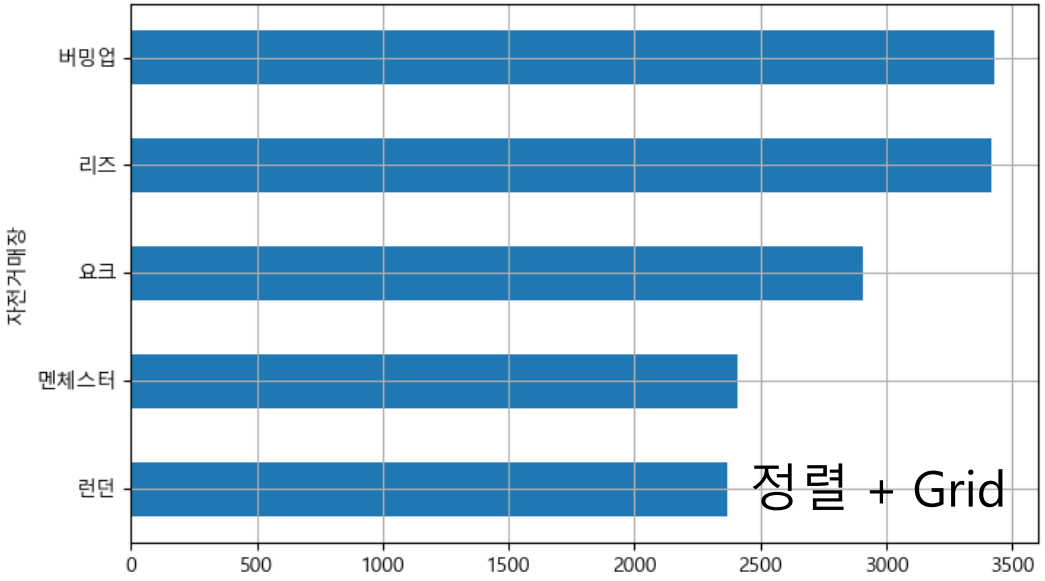
- 막대그래프의 구성요소
  - **축: 평소 축을 그대로 두는 것이 좋음**
  - **영점기준선: 제거(조정)하면 좋지 않음**
  - **헤더: 개별 막대의 범주의 식별자로 사용됨으로 그대로 두는 것이 좋음**
  - **+ 격자선**

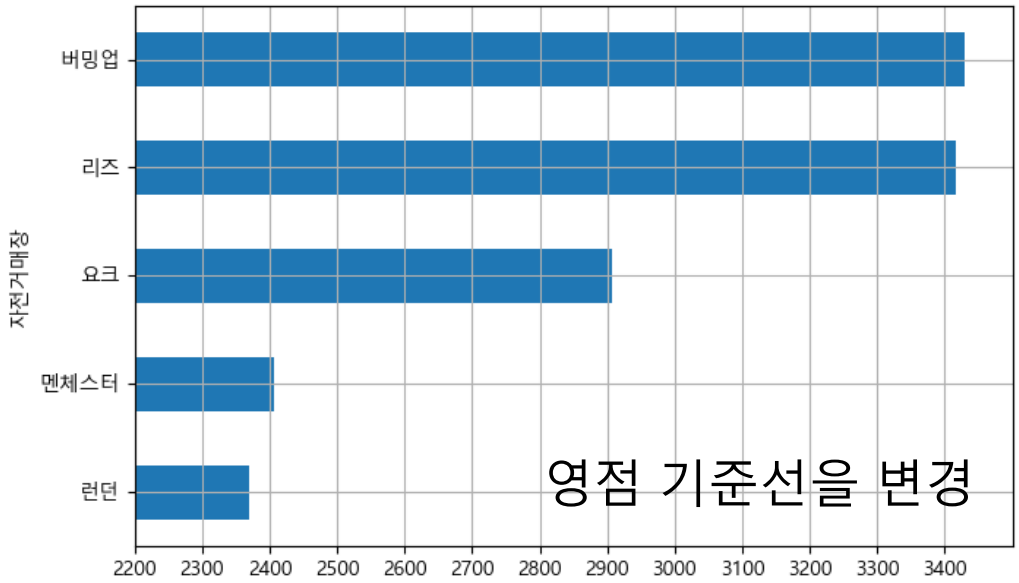


 © KOPO-Dept.
AI Engineering Limited

데이터 살펴보기

- 막대그래프 예시

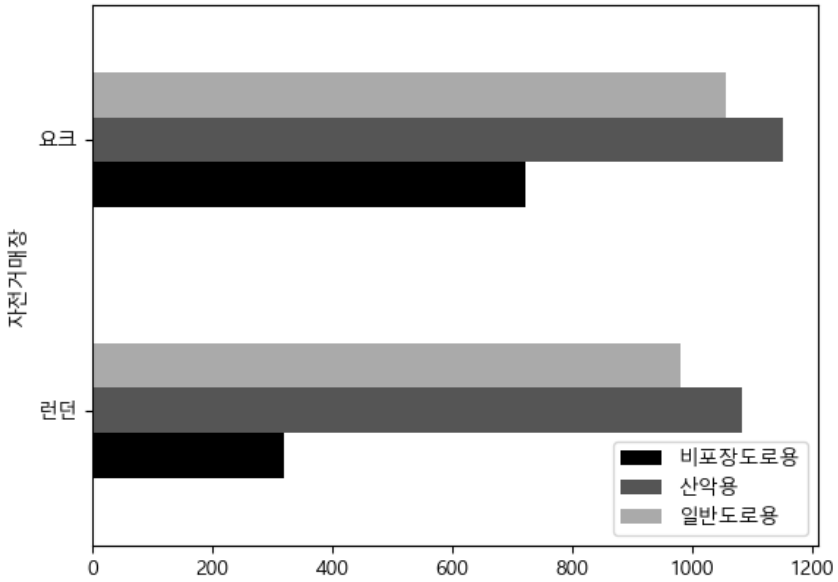


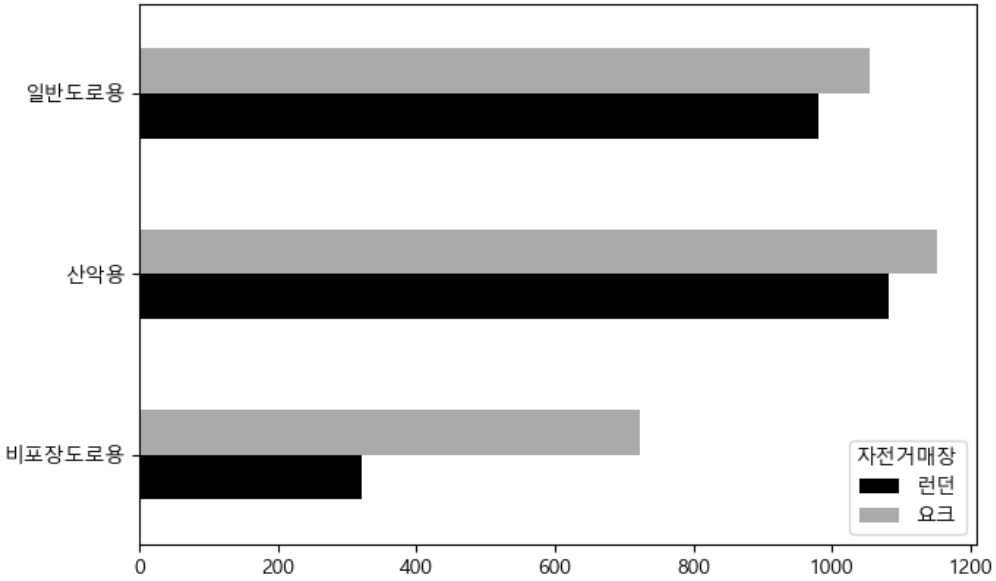


© KOPO-Dept.
AI Engineering Limited

데이터 살펴보기

- 막대그래프 예시
  - **각 매장에서, 어떤 종류의 자전거가 얼마나 주문되었나? Or (➔ 매장(지역별) 판매량)**
  - **어느 매장이 각 종류의 자전거를 가장 많이 주문했는가? (➔ 종류별 판매량)**

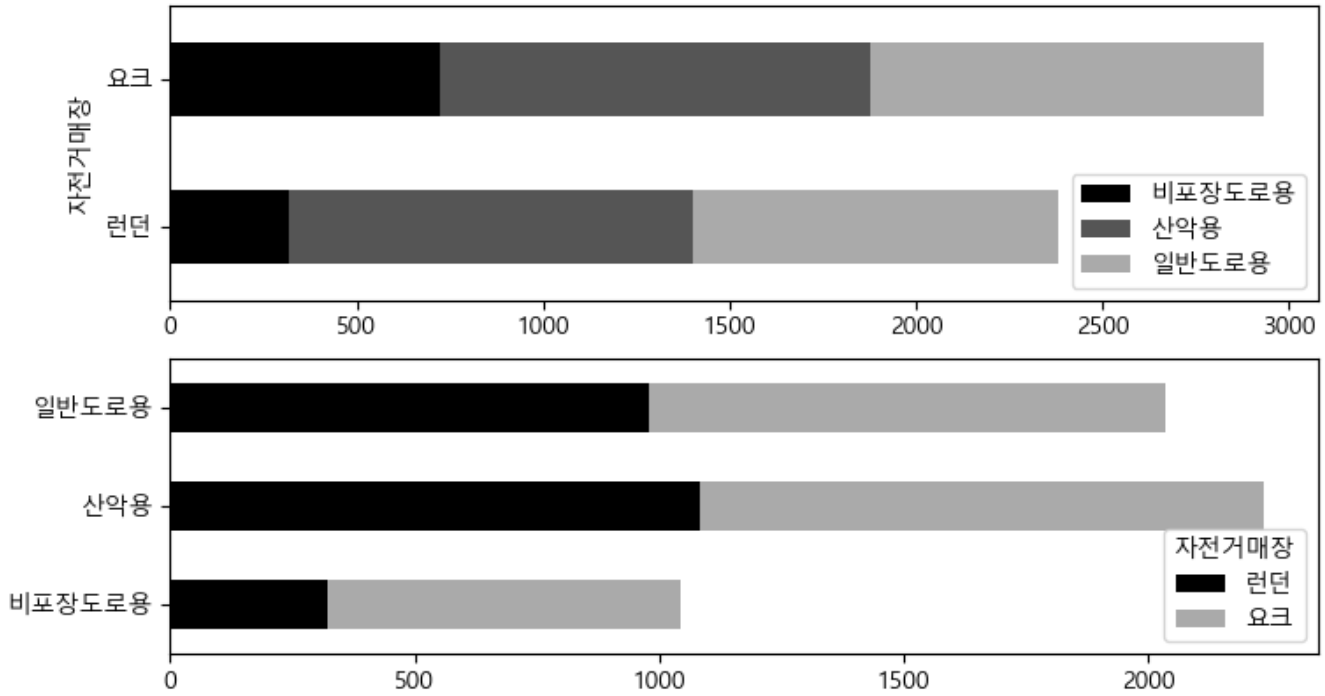




© KOPO-Dept.
AI Engineering Limited

데이터 살펴보기

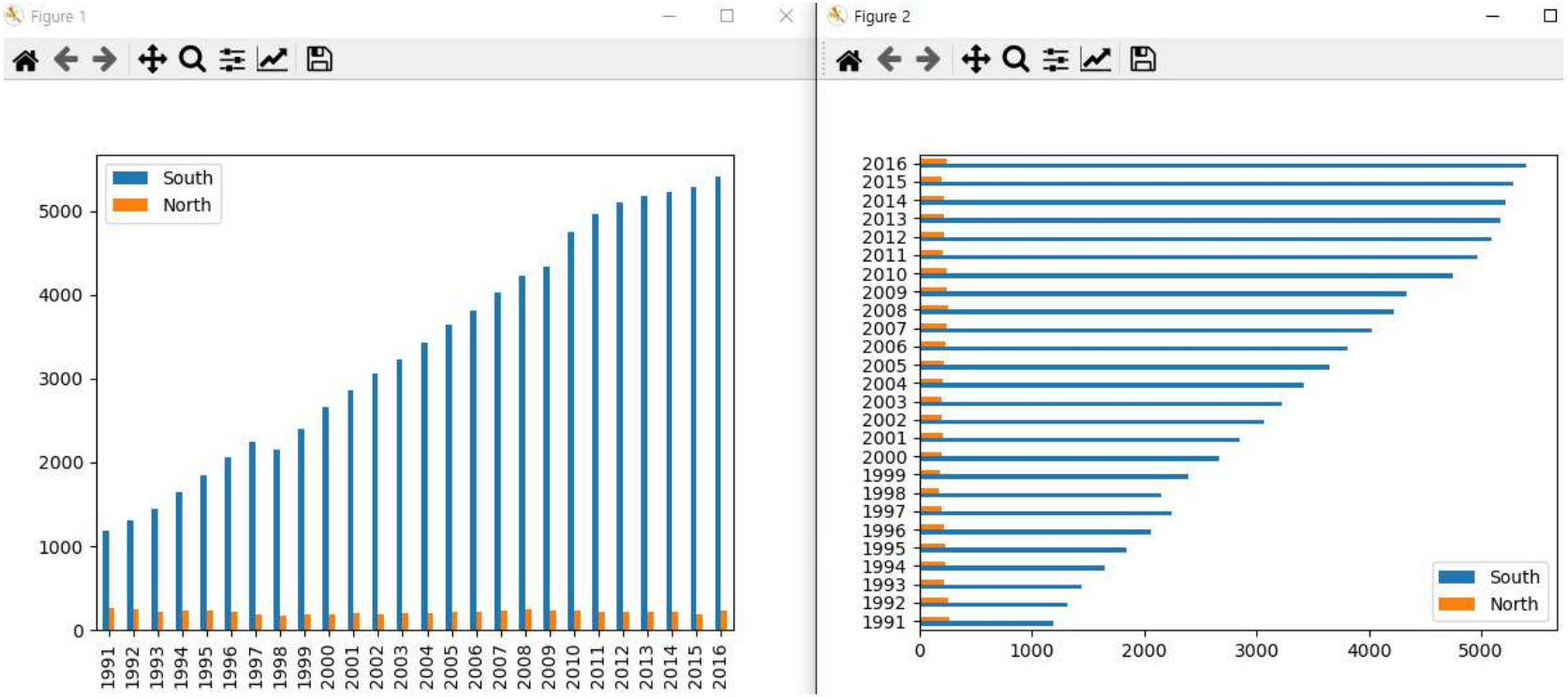
- 막대그래프 예시
  - **누적 막대 차트**



 © KOPO-Dept.
AI Engineering Limited

데이터 살펴보기

- 판다스 내장 그래프 도구 활용
  - 막대 그래프: **DataFrame.plot.bar(), DataFrame.plot.barh()**
    - **또는 DataFrame.plot(kind='bar')**



© KOPO-Dept.
AI Engineering Limited

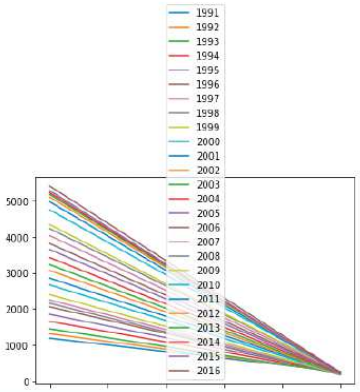
데이터 살펴보기

- 판다스 내장 그래프 도구 활용
  - 선그래프
    - Row: x축
    - Column: 범례
    - Data: y축

```
        1991  1992  1993  1994  1995 ... 2012  2013  2014  2015  2016
South   1186  1310  1444  1650  1847 ... 5096  5171  5220  5281  5404
North    263   247   221   231   230 ...  215   221   216   190   239

[2 rows x 26 columns]

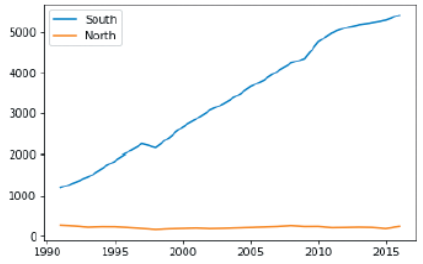

Out[1]: <matplotlib.axes._subplots.AxesSubplot at 0xa244e50>
```



Using Transpose

```
      South  North
1991   1186    263
1992   1310    247
1993   1444    221
1994   1650    231
1995   1847    230

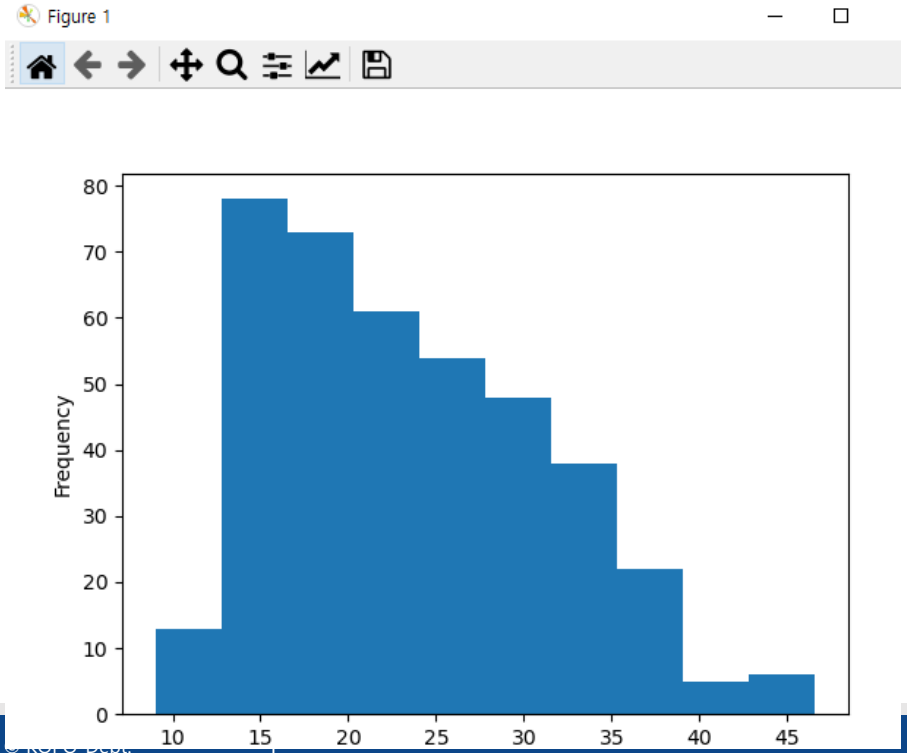

Out[2]: <matplotlib.axes._subplots.AxesSubplot at 0x17e6bb0>
```



© KOPO-Dept.
AI Engineering Limited

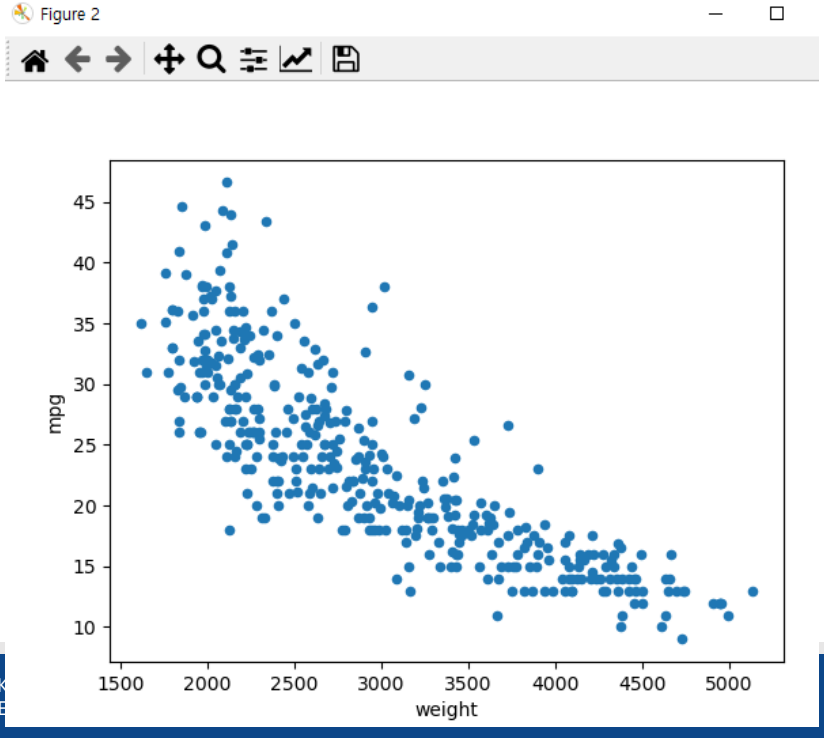
데이터 살펴보기

- 판다스 내장 그래프 도구 활용
  - 히스토그램 그래프: 구간을 나누어 해당 구간마다의 빈도수를 파악
    - mpg: 연비별 차량 대수
    - **DataFrame.plot.hist() 또는 DataFrame.plot(kind='hist')**



데이터 살펴보기

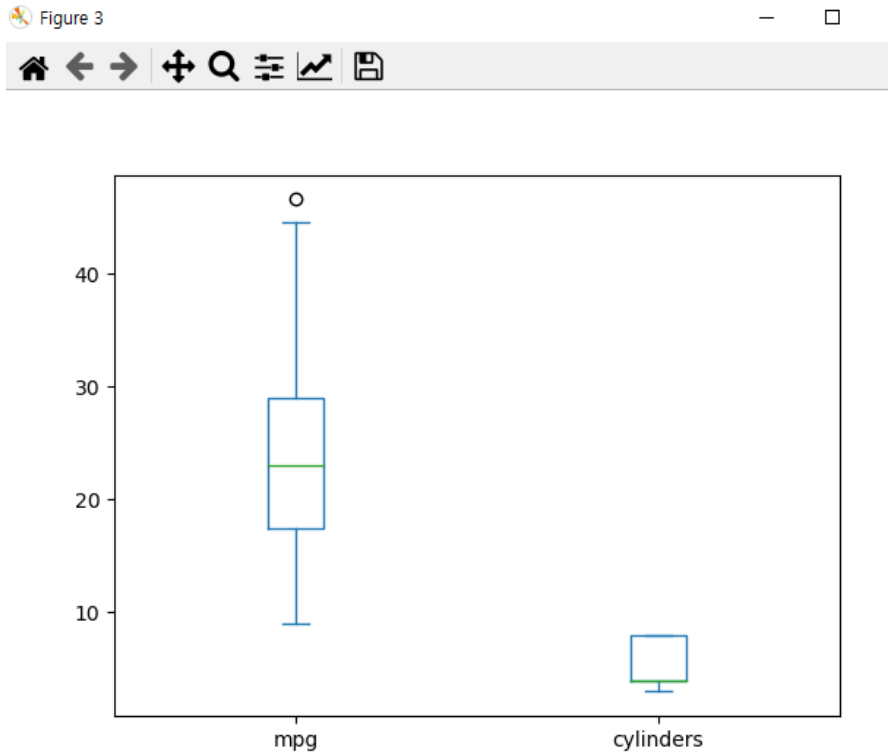
- 판다스 내장 그래프 도구 활용
  - 산점도 그래프: 두 변수의 관계를 나타내는 산점도
    - mpg 와 weight(차중)과의 관계
    - **DataFrame.plot.scatter(x='weight', y='mpg')**
    - **또는 DataFrame.plot(x='weight', y='mpg', kind='scatter')**



Daegu Campus of KOREA POLYTECHNIC

데이터 살펴보기

- 판다스 내장 그래프 도구 활용
  - 박스 플롯: 특정 변수의 데이터 분포와 분산 정도에 대한 정보를 제공
    - **DataFrame.plot.box()**
    - **또는 DataFrame.plot(kind='box')**
    - **DataFrame[['mpg', 'cylinders']].plot.box()**



데이터 살펴보기

- 실습

```
import pandas as pd

# plot 출력을 위해
import matplotlib.pyplot as plt
df = pd.read_excel('남북한발전전력량.xlsx')

print(df.info())
print(df.head(8))
df1 = df.iloc[[0, 5], 3:]
# row index: X축
# column: 범례
df1.index = ['South', 'North']
print(df1)

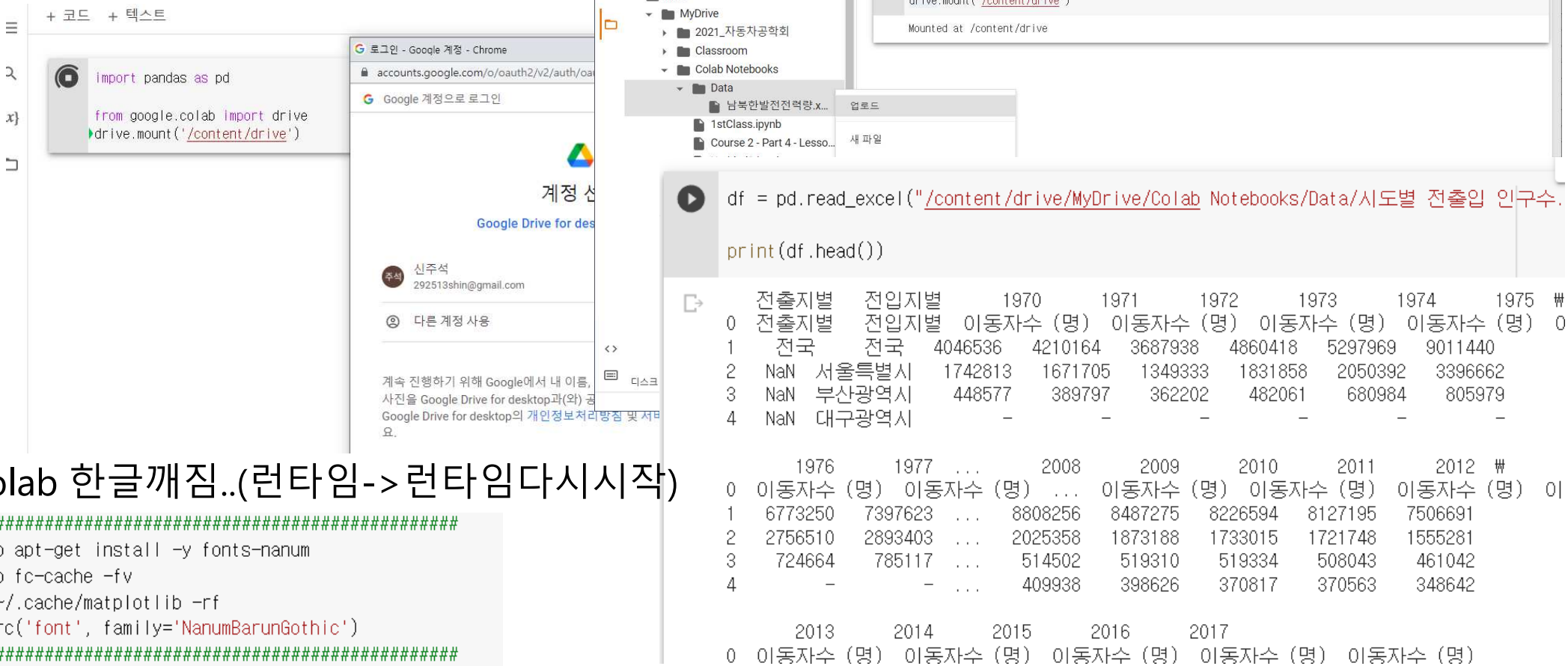
# 연도별 남북 총 발전량을 선그래프로 표현
# row index가 연도가 되어야 하고, 남한/북한이 범례가 되어야 함 (행열 변환: Transpose)
df2 = df1.T
print(df2.head(4))

# df1.plot.line()
# df2.plot()
# Bar 그래프로 표현 (plot.bar(), or plot(kind='bar'))
df2.plot.bar()  # df2.plot.barh()
```


© KOPO-Dept.
AI Engineering Limited

데이터 살펴보기

- 실습 (Colab)



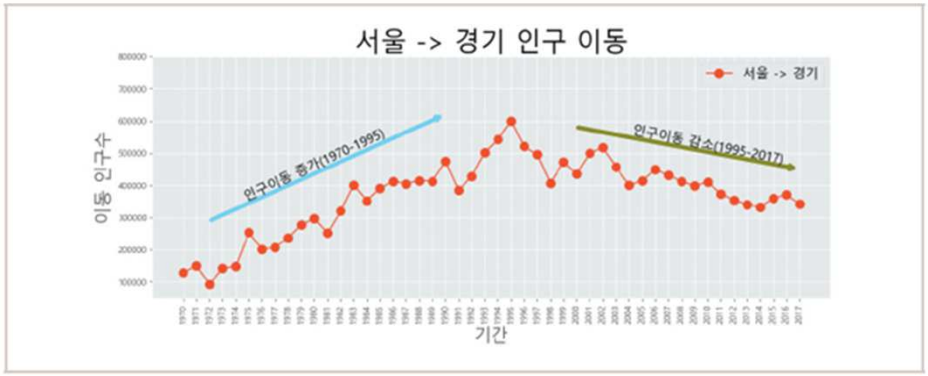
Colab 한글깨짐..(런타임->런타임다시시작)

```
##################################################
!sudo apt-get install -y fonts-nanum
!sudo fc-cache -fv
!rm ~/.cache/matplotlib -rf
plt.rc('font', family='NanumBarunGothic')
##################################################
```


© KOPO-Dept.
AI Engineering Limited

시각화 도구

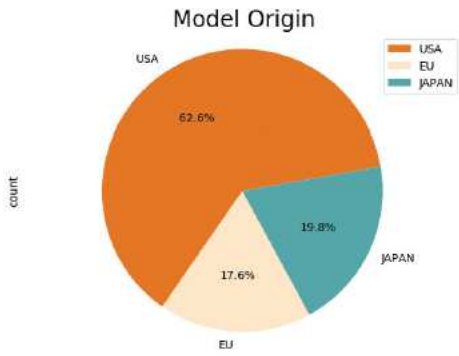
- Matplotlib – 기본 그래프 도구
  - 파이썬 표준 시각화 도구라고 부를수 있을 정도로 2D 평면 그래프에 관한 다양한 포맷과 기능을 제공
  - 선 그래프, 파이 차트 (나머지는 필요에 따라 찾아서 사용)





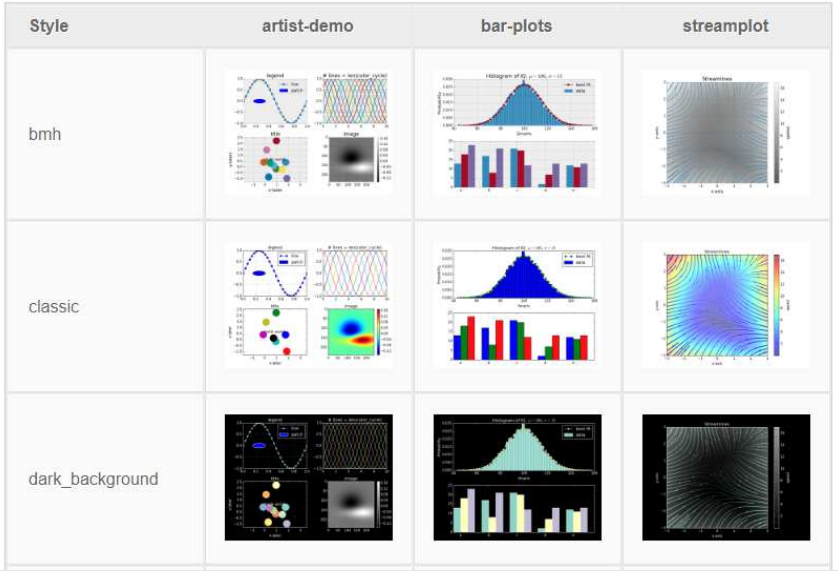
origin	mpg	cylinders	displacement	...	acceleration	model year	count
				...			
1	5000.8	1556	61229.5	...	3743.4	18827	249
2	1952.4	291	7640.0	...	1175.1	5307	70
3	2405.6	324	8114.0	...	1277.6	6118	79

[3 rows x 7 columns]



시각화 도구

- Matplotlib – 기본 그래프 도구
  - 선 그래프
    - 시도 간 인구 이동 데이터 셋 (통계청): 시도별 전출입 인구수.xlsx
      - 서울에서 다른 지역으로 이동한 데이터만 추출하여 [서울->경기도]로 이동한 인구에 대해 선 그래프를 그려보자.
      - 스타일 서식
        - » http://tonysyu.github.io/raw_content/matplotlib-style-gallery/gallery.html



범례 위치

Location String	Location Code
'best'	0
'upper right'	1
'upper left'	2
'lower left'	3
'lower right'	4
'right'	5
'center left'	6
'center right'	7
'lower center'	8
'upper center'	9
'center'	10

© KOPO-Dept.
AI Engineering Limited

시각화 도구

- Matplotlib – 기본 그래프 도구
  - 선 그래프
    - 선 그래프 꾸미기 옵션 (example)

꾸미기 옵션	설명
'o'	선 그래프가 아니라 점 그래프로 표현
marker='o'	마커 모양 (예: 'o', '+', '*', '.')
markerfacecolor='green'	마커 배경색
markersize=10	마커 크기
color='olive'	선의 색
linewidth=2	선의 두께
label='서울 -> 경기'	라벨 지정

© KOPO-Dept.
AI Engineering Limited

시각화 도구

- Matplotlib – 기본 그래프 도구
  - 선 그래프
    - 선 그래프 (**Annotation**/Comment): annotate() 함수 사용
    - arrowprops 옵션: 텍스트 대신 화살표가 표시
    - comment를 넣은 공간을 확보하기 위해 ylim()함수를 사용하여 y축 범위 지정
    - annotate 함수에서 사용되는 x, y 위치: x는 인덱스 번호, y는 y좌표값에 들어갈 인구수 데이터
      - xy=(20, 620000): 인덱스 번호 20을 x값으로 하고, 620000명을 y값으로 한다는 뜻.
      - rotation: +는 양의 기울기(반시계방향), -는 음의 기울기(시계방향)
      - va: 글자를 위아래 세로 방향으로 정렬: center, top, bottom, baseline
      - ha: 글자를 좌우로 정렬: center, left, right

1) verticalalignment

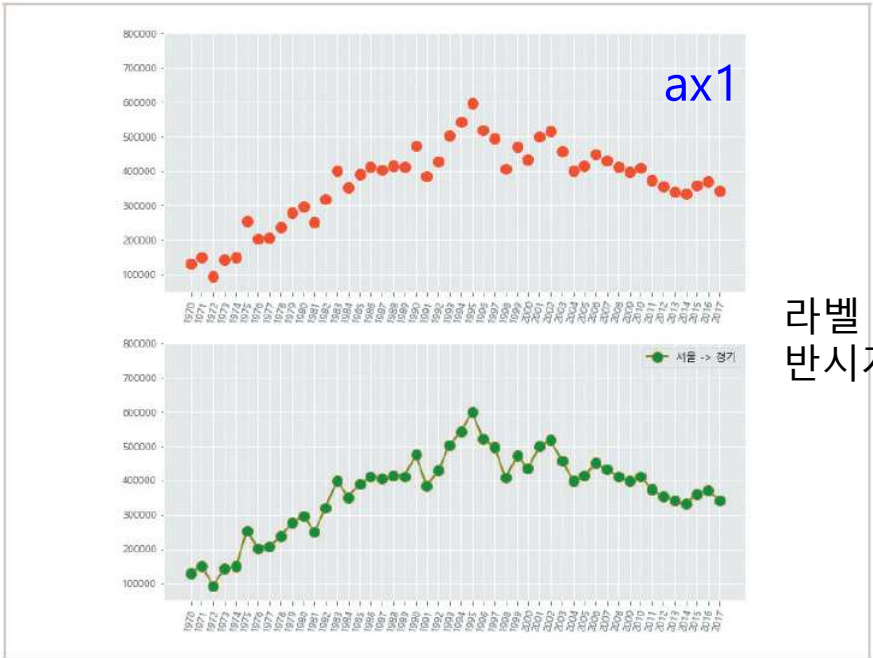
- center : 수직방향으로 좌표 중앙에 놓임
- top : 좌표가 텍스트 위에 놓임
- bottom : 좌표가 텍스트 아래 놓임
- baseline : 텍스트의 baseline 에 따라 달라짐

2) horizontal alignment

- center : 수평방향으로 좌표 중앙에 놓임
- left : 좌표가 텍스트 왼쪽에 놓임
- right : 좌표가 텍스트 오른쪽에 놓임

시각화 도구

- Matplotlib – 기본 그래프 도구
  - 선 그래프
    - **화면을 분할하여 그래프 여러 개** 그리기: add_subplot() 메소드 사용
      - **add_subplot**(행의 크기, 열의 크기, 서브플롯 순서)
        - » ex: ax1 = fig.add_subplot(2, 1, 1) [2행 1열 중 첫번째 객체를 나타냄]

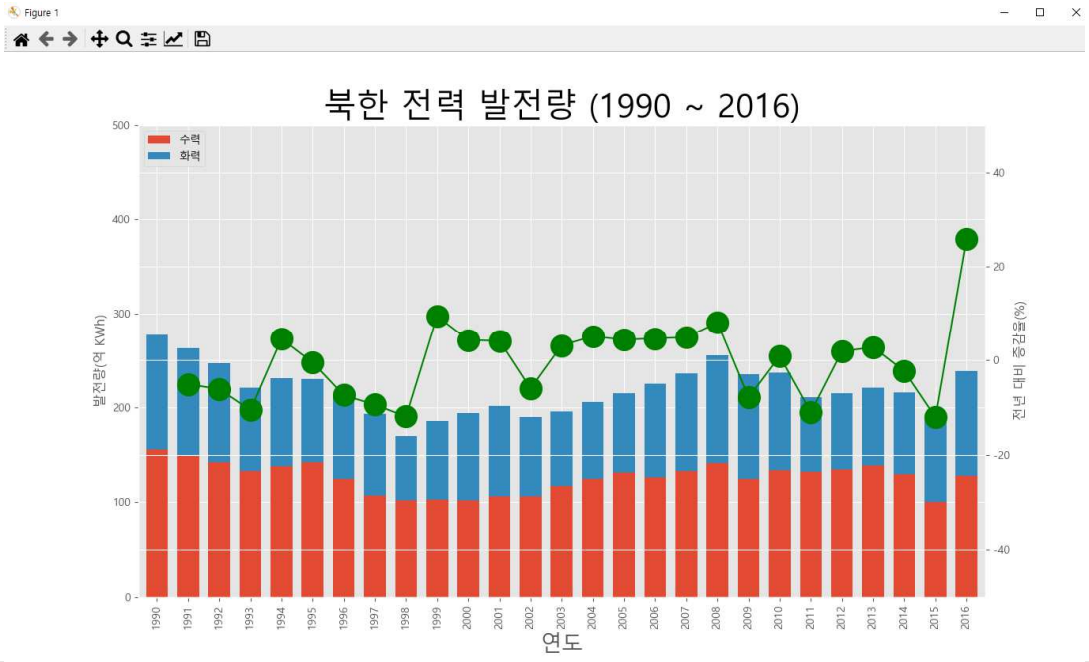


라벨 기울기: ax.set_xticketlabels(series, rotation=75)
반시계방향

© KOPO-Dept.
AI Engineering Limited

시각화 도구

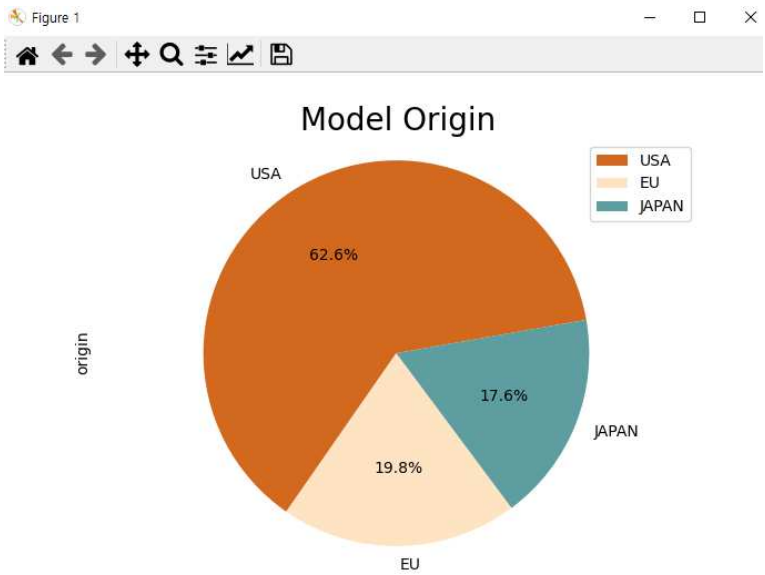
- Matplotlib – 기본 그래프 도구
  - 선 그래프
    - 보조축 활용하기(2축 그래프 그리기(x축을 공유: twinx()))
      - 막대그래프: 북한의 화력, 수력 전력량
      - 선 그래프: 북한 발전량의 "전년 대비 증감률"[=(현재년도-전년도/전년도)*100]



© KOPO-Dept.
AI Engineering Limited

시각화 도구

- Matplotlib – 기본 그래프 도구
  - 파이 차트
    - 조각의 크기는 해당 변수에 속하는 데이터 값의 크기에 비례
    - plolt() 메소드에 kind='pie' 옵션을 사용
      - auto-mpg 데이터셋에서 제조국(origin)을 counting하여 파이차트로 표현
        - » groupby('origin').sum() 사용
        - » value_count() 사용



 © KOPO-Dept.
AI Engineering Limited

데이터 살펴보기

- 실습

```
# 스타일 적용
print(plt.style.available)
plt.style.use('ggplot')
# plt.style.use('seaborn-deep')

# Excel 데이터를 데이터프레임 변환
df = pd.read_excel('시도별 전출입 인구수.xlsx', engine='openpyxl', header=0)
print(df.head())

# 서울에서 다른 지역(경기도)으로 이동한 데이터만 추출하여 정리
# 누락값(NaN)을 앞 데이터로 채움 (엑셀 양식 병합 부분)
# df1 = df.fillna(method='bfill')
# print(df1.head())
# NaN를 채우는 방법 (ffill: NaN 나오기 바로 이전의 데이터를 복사해서 사용, bfill: NaN이 나오고, 다음 나오
df = df.fillna(method='ffill')
print(df.head())

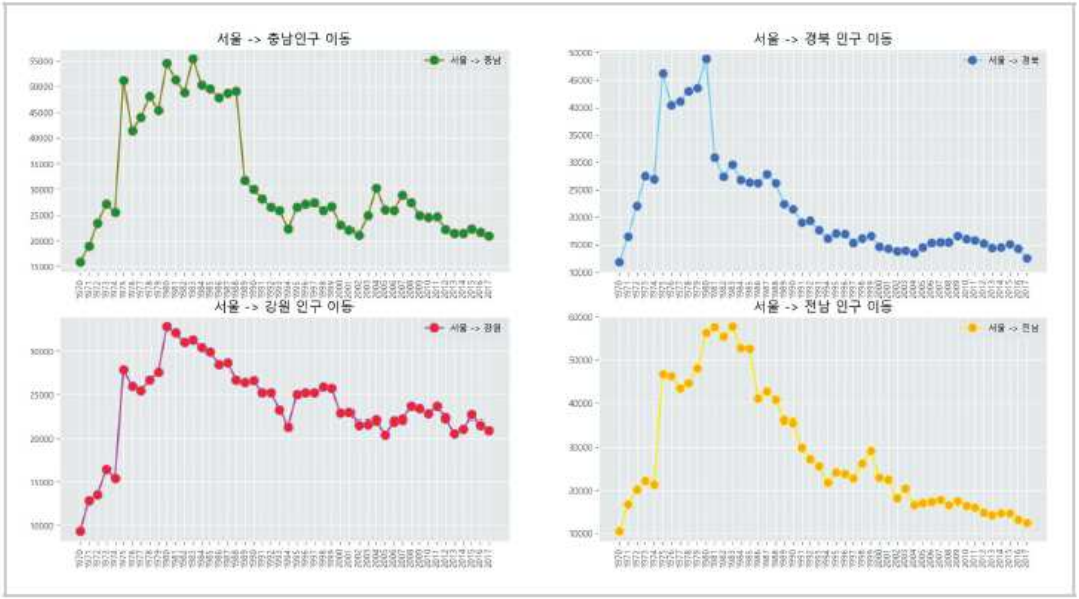
# 서울 ==> 서울은 고려할 필요가 없으므로..
mask = (df['전출지별'] == '서울특별시') & (df['전입지별'] != '서울특별시')
# print(mask[19:19+18])
# mask가 True인 것만.
df_seoul = df[mask]
```

시각화 도구

- Matplotlib – 기본 그래프 도구
  - 선 그래프



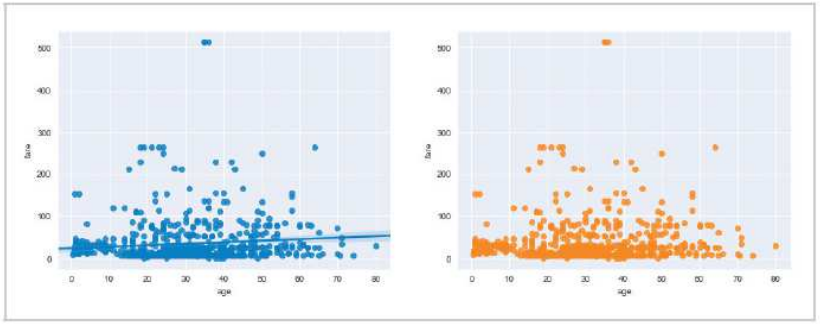
실습 해보기

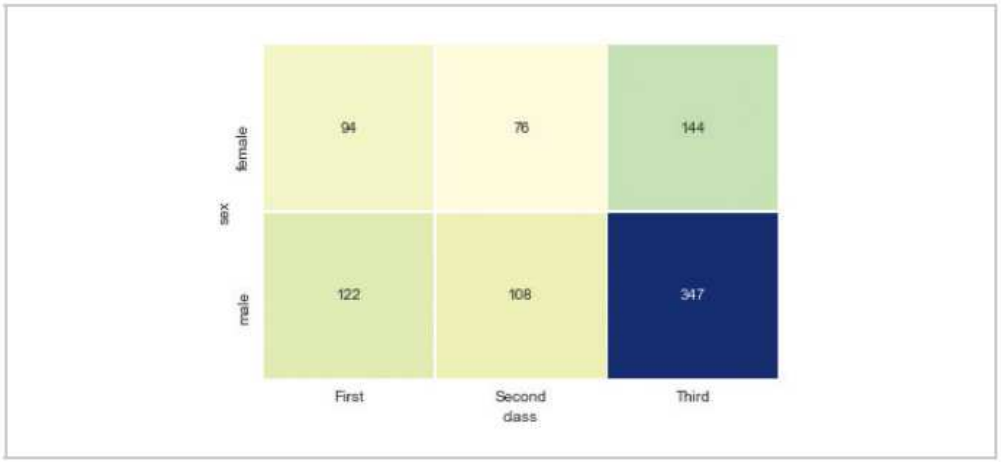


© KOPO-Dept.
AI Engineering Limited

시각화 도구

- Seaborn – 고급 그래프 도구
  - Matplotlib의 기능과 스타일을 확장한 파이썬 시각화 도구의 고급 버전
    - 아나콘다 배포판의 경우, 기본으로 설치되어 있음
    - Seaborn 라이브러리의 alias name은 주로 sns로 많이 사용 (import seaborn as sns)
    - Regression 포함된 산점도 그래프, 히트맵 실습 (vs. matplotlib, numpy)

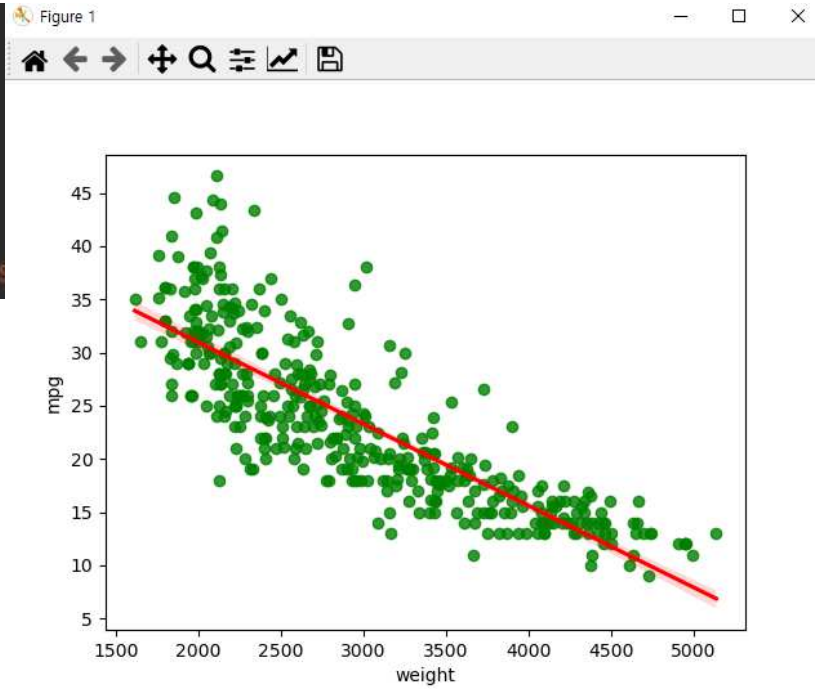




시각화 도구

- Seaborn – 고급 그래프 도구
  - Regression 포함된 산점도 그래프
    - auto-mpg 데이터셋에서 weight, mpg로 산점도 그래프
    - Seaborn 라이브러리의 **regplot() 메소드**를 이용하여 산점도 그래프 위에 그리기

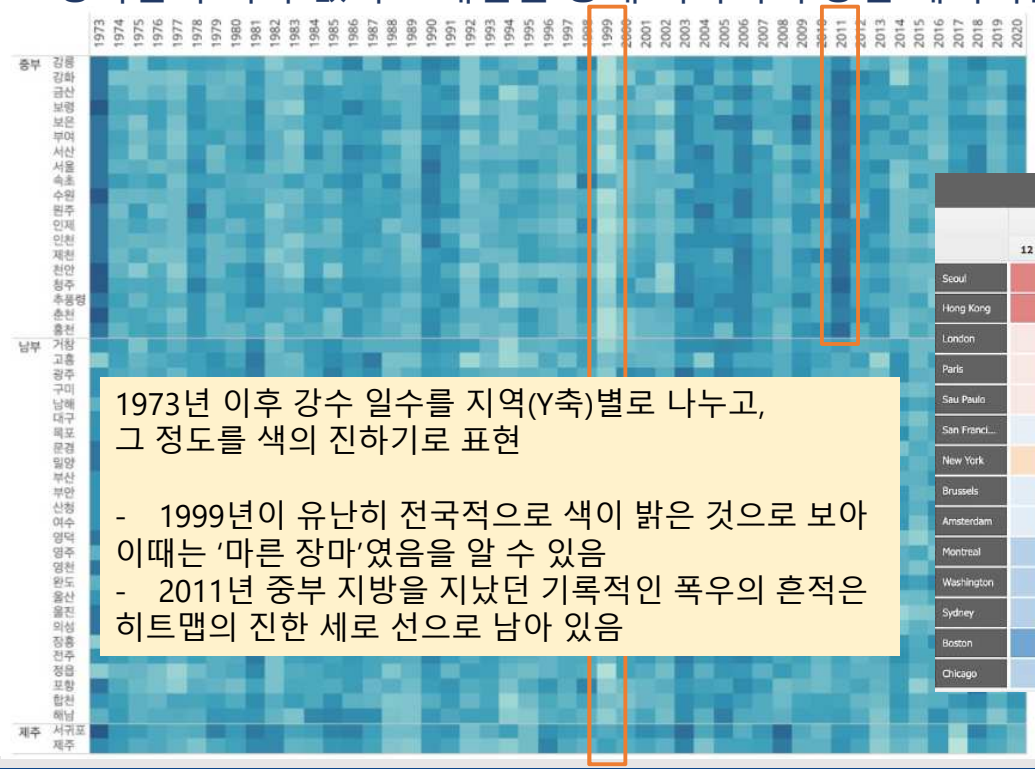
```
# seaborn을 이용한 산점도 + Regression Line
# weight와 mpg 관계
df_scatter = df[['weight', 'mpg']]
print(df_scatter.head())
sns.regplot(x='weight', y='mpg', data=df_scatter,
            scatter_kws={"color": "green"}, line_kws
```



Daegu Campus of KOREA POLYTECHNIC

시각화 도구

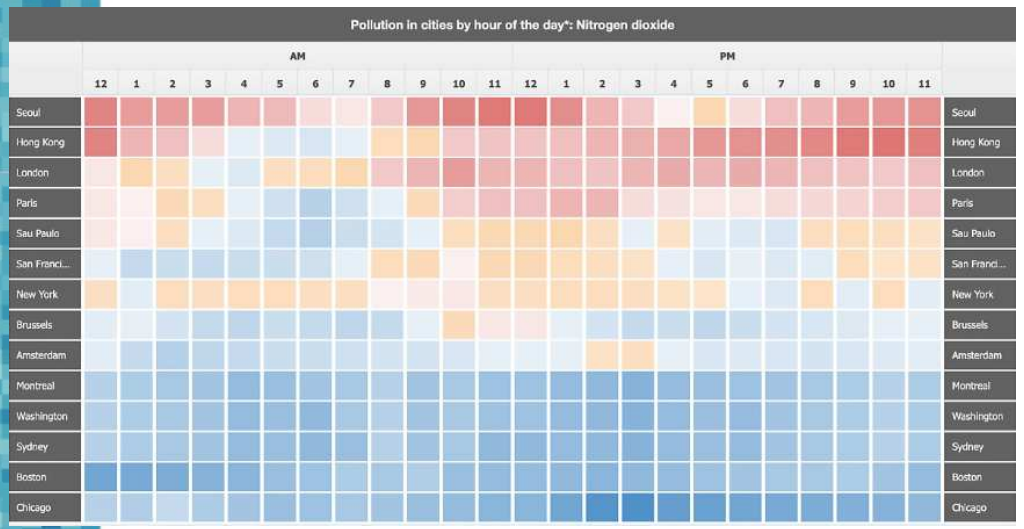
- Seaborn – 고급 그래프 도구
  - 히트맵 그래프
    - X축과 Y축을 범주형 변수로 하고, 균일한 블록으로 나누어 각 칸에 수치형 변수를 채우는 방식
    - 정확한 수치가 없어도 패턴을 통해 시각화의 중심 메시지를 한눈에 전할 수 있는 그래프



1973년 이후 강수 일수를 지역(Y축)별로 나누고,
그 정도를 색의 진하기로 표현

- 1999년이 유난히 전국적으로 색이 밝은 것으로 보아 이때는 '마른 장마'였음을 알 수 있음
- 2011년 중부 지방을 지났던 기록적인 폭우의 흔적은 히트맵의 진한 세로 선으로 남아 있음

도시별 시간에 따른 공기 질



 © KOPO-Dept.
AI Engineering Limited

시각화 도구

- Seaborn – 고급 그래프 도구
  - 히트맵 그래프
    - 연도(year)별 월(month)별 승객 수(passengers)를 모아놓은 flights 라는 seaborn에 내장되어 있는 데이터프레임 사용
    - **pivot() 메소드**를 이용하여 월별 연도별 승객수를 집계한 피벗 테이블(pivot table)을 만들어서, 이를 이용해서 히트맵 그려보기

	year	month	passengers
0	1949	January	112
1	1949	February	118
2	1949	March	132
3	1949	April	129
4	1949	May	121

pivot (month, year, passengers)



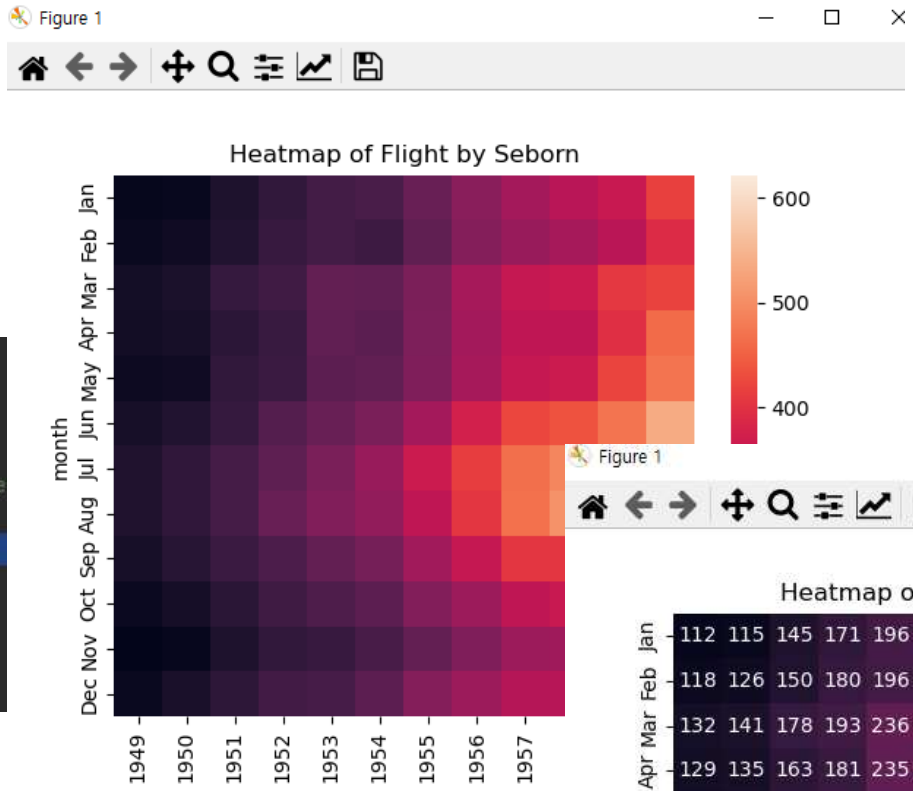
year	1949	1950	1951	1952	1953	1954	1955	1956	1957	1958	1959	1960
month												
January	112	115	145	171	196	204	242	284	315	340	360	417
February	118	126	150	180	196	188	233	277	301	318	342	391
March	132	141	178	193	236	235	267	317	356	362	406	419
April	129	135	163	181	235	227	269	313	348	348	396	461
May	121	125	172	183	229	234	270	318	355	363	420	472

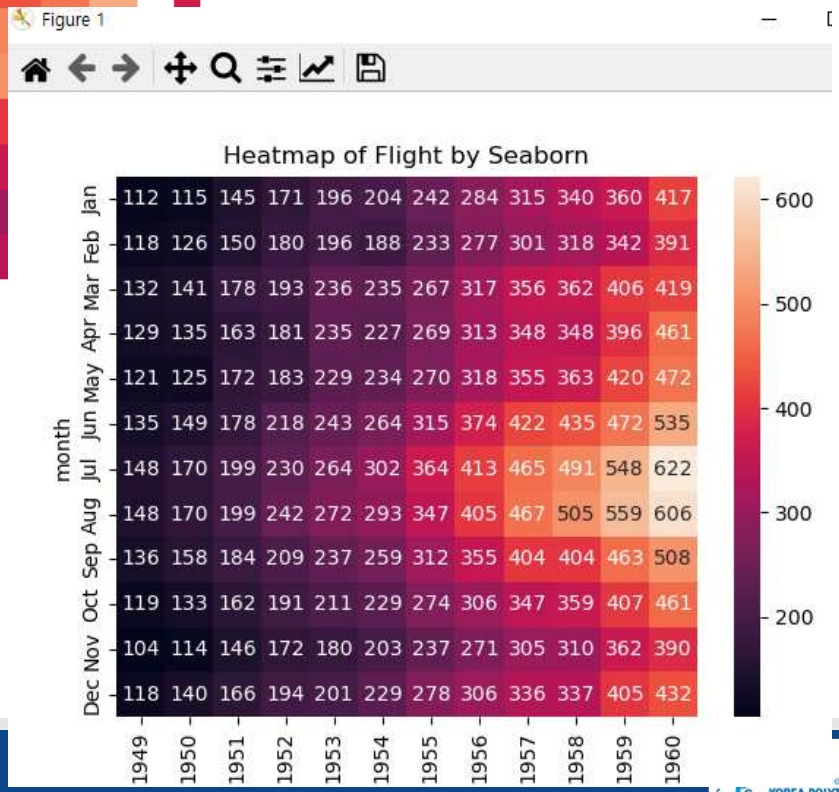
© KOPO-Dept.
AI Engineering Limited

시각화 도구

- Seaborn – 고급 그래프 도구
  - 히트맵 그래프

```
# seaborn heatmap 그리기
# Dataset 읽기
flights_data = sns.load_dataset("flights")
print(flights_data.head())
df = flights_data.pivot('month', 'year', 'passenge
print(df.head())
# fig = plt.figure()
# ax = fig.add_subplot(1, 1, 1)
ax = sns.heatmap(df)
plt.title("Heatmap of Flight by Seborn")
plt.show()
```





Thank you

Q&A

www.kopo.ac.kr
jsshin7@kopo.ac.kr

 © KOPO-Dept.
AI Engineering Limited